Part 1：OpenClaw 源码满血版下载部署

> *从零开始，15 分钟内在你的 Mac 上安装并运行一个拥有 32 万 GitHub Star 的开源 AI 助手操作系统。*

引言

你想不想拥有一个比 Siri 聪明 100 倍的私人 AI 助手？

它能记住你说过的每一句话，能帮你操作电脑上的文件、浏览网页、分析数据，甚至能同时连接微信、飞书、Discord 等 20 多个聊天软件。更重要的是——它完全运行在你自己的电脑上，你的数据一个字节都不会泄露出去。

这个助手就是 **OpenClaw**，一个拥有 **321,000+ GitHub Star** 的开源 AI 助手操作系统。它不是又一个"ChatGPT 套壳"——而是一个真正运行在你本地的、可定制的、可编程的智能助手平台。今天，我们就从零开始，把它装到你的 Mac 上。

整个安装过程大约需要 15 分钟。完成之后，你会在终端上看到 OpenClaw 成功运行的第一个画面——那将是你拥有私人 AI 助手的起点。

1.1 OpenClaw 项目介绍

OpenClaw 是什么

OpenClaw 的全名是 **Open Claw**（开源之爪），口号是：

> *"Your own personal AI assistant. Any OS. Any Platform. The lobster way."*
>
> *（你的私人 AI 助手。任何操作系统，任何平台。龙虾之道。）*

它是一个**本地优先的个人 AI 助手操作系统**。注意，不是"聊天机器人"，不是"API 套壳"，而是一个完整的**操作系统**——就像 macOS 管理你的电脑一样，OpenClaw 管理你的 AI 助手。

来看一组关键数据：

项目	详情
GitHub Stars	**321,000+**（截至 2026 年 3 月）
开源协议	MIT（完全免费，可商用）
开发语言	JavaScript / TypeScript
支持平台	macOS、Linux、Windows（WSL2）
消息通道	WhatsApp、Telegram、Discord 等 **20+** 种
内置技能	**100+** 个（浏览器自动化、文件操作、代码执行等）
当前最新版	**2026.3.13**

OpenClaw 的核心能力可以用一张表来概括：

核心能力	说明
Gateway 架构	统一的 WebSocket 控制面，管理所有功能
多通道接入	同时对接 20+ 即时通讯软件
持久记忆	跨会话记住你是谁、你在做什么
100+ 内置技能	浏览器操作、文件处理、代码执行、音乐控制...
Cron 定时任务	自动执行后台任务
语音交互	语音唤醒 + 对话模式
Skills 生态	社区技能市场（ClawHub）13,000+ 扩展

和你知道的 AI 工具有什么不同？

你可能已经用过 Siri、ChatGPT 或者 Dify，它们和 OpenClaw 有什么区别？一张表说清楚：

维度	SIRI / 小爱同学	CHATGPT / CLAUDE 网页版	DIFY / COZE	OPENCLAW
运行位置	云端	云端	云端 / 自托管	你的电脑上
数据隐私	数据上传苹果/小米	数据在 OpenAI	取决于部署	全部在本地
可定制性	几乎不能改	不能改	可视化搭建	完全可编程
持久记忆	无	有限	有限	**Markdown** 文件，随便改
多渠道	仅语音	仅网页/App	API	**20+** 消息渠道
系统操作	受限	不能	不能	读写文件、执行命令、浏览网页
定位	语音助手	聊天工具	应用开发平台	个人 **AI** 助手操作系统

简单来说：**ChatGPT** 是租来的公寓，**OpenClaw** 是你自己装修的别墅——你想怎么改就怎么改，钥匙在你手里。

发展历程

OpenClaw 的发展历程也值得一提：

- **前身**：ClawdBot → MoltBot → 统一为 OpenClaw
- **2026** 年里程碑：正式升级为 "Agent Operating System"（智能体操作系统）
- **NVIDIA** 合作：基于 OpenClaw 推出企业级 **NemoClaw**
- **OOLONG** 基准测试：得分 **74.8**，超越 Claude Code 的 70.3

这不是一个玩具项目——它背后有庞大的开源社区和企业级支持。

1.2 OpenClaw 最新版本与功能对比

版本号命名规则

OpenClaw 采用日历版本（**Calendar Versioning**）命名：格式为 `vYYYY.M.D`。

比如 `2026.3.13` 的含义是：**2026 年 3 月 13 日**发布的版本。这种命名方式的好处是，你一看版本号就知道这个版本有多新。

OpenClaw 有三个更新频道：

频道	说明	适合人群
stable	稳定版（默认）	所有用户
beta	测试版	尝鲜用户
dev	开发版	贡献者

最新三个版本亮点

版本	发布日期	核心亮点
2026.3.13（当前最新）	2026-03-14	修复 compaction token 计数、Telegram SSRF 安全处理、Android UI 改版
2026.3.12	2026-03-13	**Dashboard v2**（模块化视图 + 命令面板）、Provider-plugin 架构重构
2026.3.7	2026-03	支持 GPT-5.4、OOLONG 基准测试 74.8 分

可以看到，OpenClaw 的迭代速度非常快——**平均每 2-3 天就有一个新版本**。这也意味着你需要掌握升级方法（我们会在 Part 7 中详细讲解）。

1.3 Node.js 介绍与安装

为什么需要 Node.js？

安装 OpenClaw 之前，你的 Mac 上必须先装好 **Node.js**。

Node.js 是 JavaScript 的运行环境。OpenClaw 是用 JavaScript/TypeScript 编写的，所以它需要 Node.js 才能运行。

打个比方：**Node.js 就像"发动机"，OpenClaw 就像"汽车"——没有发动机，汽车跑不起来。**

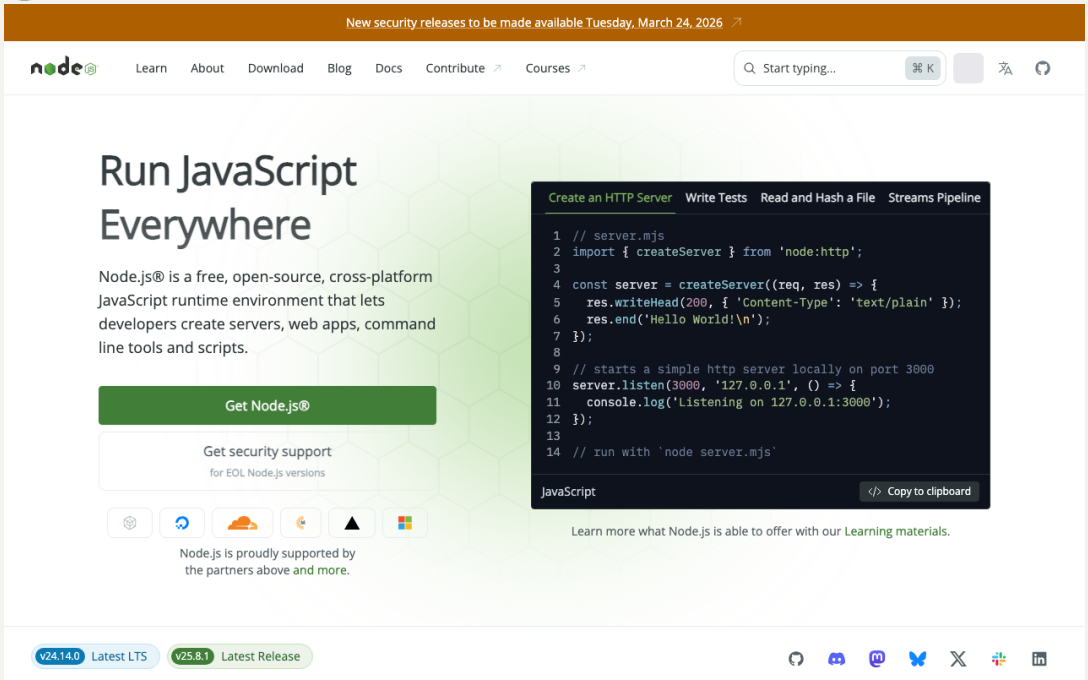
版本要求：**Node.js v24（推荐）** 或 **v22 LTS（22.16+）**。

三种安装方式

安装方式	适合人群	优点	缺点
官网安装器（推荐新手）	完全零基础	一键安装、图形界面	版本管理不便
Homebrew	Mac 用户	`brew install node` 一条命令	版本可能不是最新
nvm	开发者	多版本并存管理	配置稍复杂

如果你是第一次接触编程，推荐使用官网安装器——访问 nodejs.org，下载 macOS 安装包，双击运行安装向导，一路点"下一步"即可。

界面截图：Node.js 官网下载页面



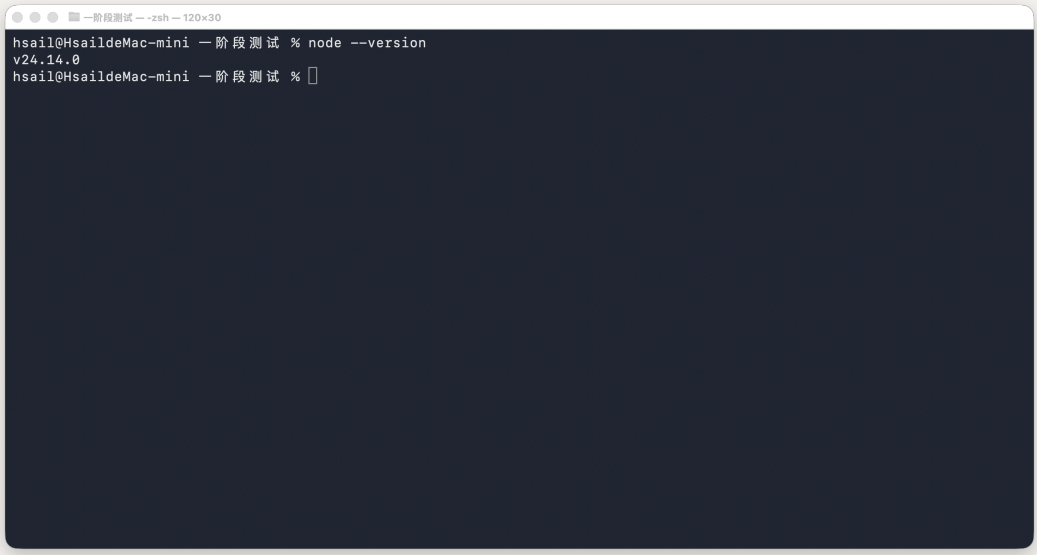
验证 Node.js 安装

安装完成后，打开终端，运行以下两条命令验证：

```
$ node --version
v24.14.0

$ npm --version
11.9.0
```

看到版本号输出，说明 Node.js 和 npm（Node.js 的包管理器）都安装成功了。

> 📸 *终端截图：验证 Node.js 和 npm 版本*
>
> 
>

如果你看到 `command not found` 的报错，说明安装没有成功——回到上面的步骤重新安装。

1.4 命令行介绍与打开方法

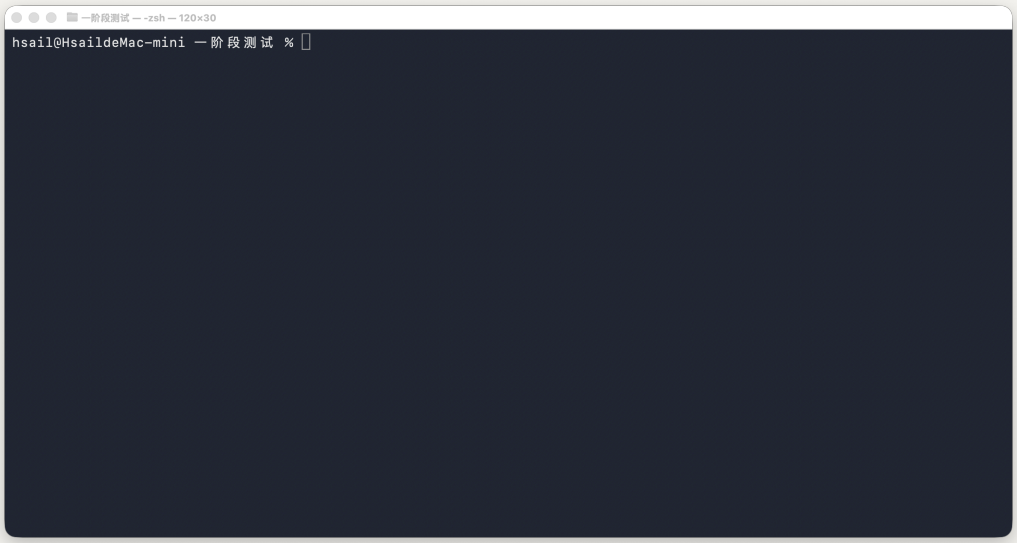
什么是命令行？

如果你从来没有用过命令行，别担心——它比你想象的简单。

命令行就是和电脑"用文字对话"的窗口。你打一行指令，电脑执行一个操作。就像你对 Siri 说"帮我打开备忘录"，只不过命令行是用键盘打字而不是说话。

在 macOS 上，命令行工具叫做 **Terminal**（终端）。打开它最快的方式：

1. 按下 `Command + Space` 打开 Spotlight 搜索
2. 输入 **Terminal**
3. 回车



你会看到一个黑色（或白色）的窗口，里面有一行光标在闪烁——这就是命令行，等着你输入指令。

基础命令速览

后面的操作中你会用到一些基本命令，先认识一下：

命令	作用	示例
`cd`	切换目录	`cd ~/Desktop`（进入桌面）
`ls`	列出当前目录的文件	`ls`（看看有什么文件）
`pwd`	显示当前所在路径	`pwd`（我在哪？）
`which`	查找命令在哪里	`which node`（node 装在哪？）
`echo $PATH`	查看环境变量 PATH	`echo $PATH`（电脑去哪找程序？）

不需要背下来，用到的时候回来查就行。

什么是 PATH 环境变量？

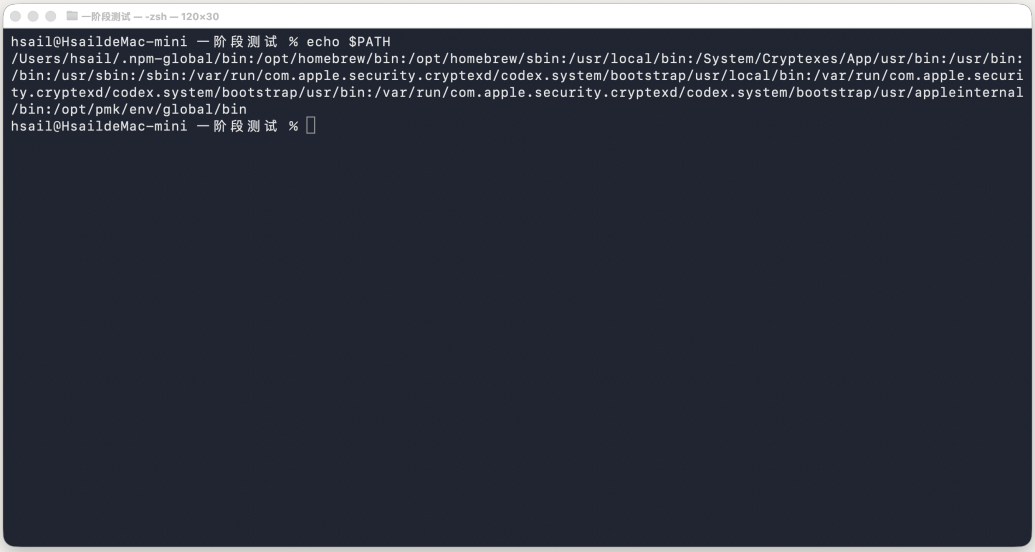
这个概念非常重要，因为后面安装 OpenClaw 时你会直接用到它。

PATH 就像电脑的"黄页"——当你在命令行输入一个命令时，电脑会按照 PATH 中列出的路径，一个一个去找对应的程序。如果所有路径都找遍了还没找到，就会报错：

```
command not found
```

来看一下你的 Mac 上 PATH 里有哪些路径：

```
$ echo $PATH
/Users/hsail/.local/bin:/opt/homebrew/bin:/opt/homebrew/sbin:/usr/local/bin:
/System/Cryptexes/App/usr/bin:/usr/bin:/bin:/usr/sbin:/sbin:..
.
```

> 📸 *终端截图：查看 PATH 环境变量*
>
> 
>

看到那一长串用 `:` 分隔的路径了吗？每个路径都是一个"搜索目录"。当你输入 `node` 命令时，电脑会依次去这些目录里找 `node` 程序。

比如我们可以验证一下 `node` 命令的位置：

```
$ which node
/usr/local/bin/node

$ which npm
/usr/local/bin/npm
```

`/usr/local/bin` 正好在 PATH 列表中，所以 `node` 和 `npm` 命令能被找到。

记住这个概念——等下安装 OpenClaw 时，你会遇到一个"经典坑"，而解决方案就是修改 PATH。

1.5 OpenClaw 下载与安装

终于到了本章的核心——安装 OpenClaw。

三种安装方式

安装方式	命令	适合人群
一键安装脚本	`curl -fsSL https://openclaw.ai/install.sh | bash`	新手首选
npm 全局安装	`npm install -g openclaw@latest`	开发者推荐
Git 源码安装	`git clone ...` + `pnpm install`	高级用户

一键安装脚本会自动检测环境、安装 OpenClaw、启动配置向导——但为了让你更好地理解安装过程，我们选择 **npm 全局安装**，手动走一遍完整流程。

实操：使用 npm 安装 OpenClaw

打开终端，输入以下命令：

```
$ npm install -g openclaw@latest
```

这条命令的含义是：

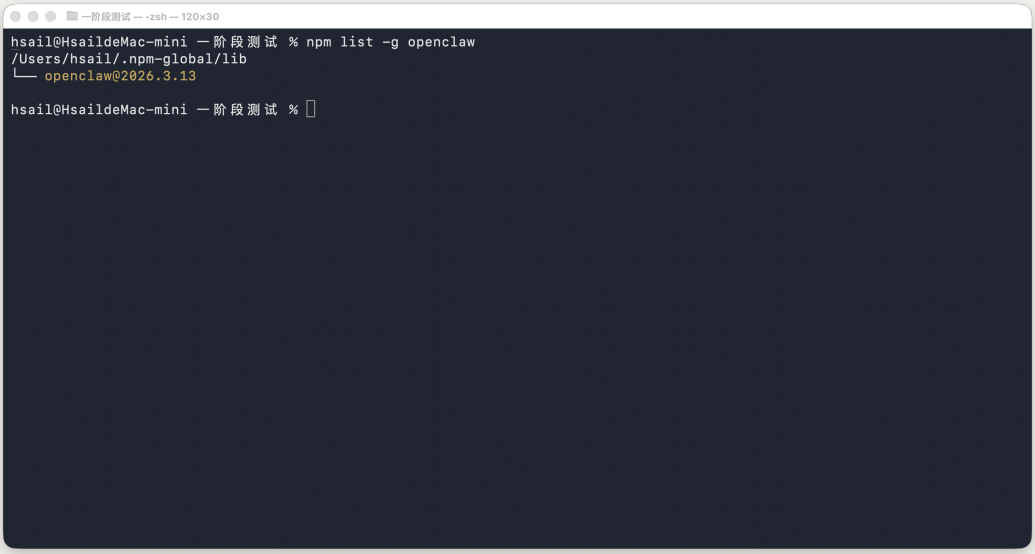
- `npm`：Node.js 的包管理器
- `install`：安装
- `-g`：全局安装（整台电脑都能用）
- `openclaw@latest`：安装 OpenClaw 的最新版本

按下回车，等待安装完成。你会看到类似这样的输出：

```
$ npm install -g openclaw@latest
npm warn deprecated node-domexception@1.0.0: Use your
platform's native DOMException instead

added 541 packages in 32s

89 packages are looking for funding
  run `npm fund` for details
```

> 📸 *终端截图：npm install 安装过程*
>
> 
>

看到 `added 541 packages in 32s`，说明安装成功了——541 个软件包，32 秒搞定。

经典坑：command not found

安装完成了，迫不及待想试试？在终端输入：

```
$ openclaw
```

然后你会看到——

```
$ openclaw
zsh: command not found: openclaw
```

什么？刚装完就找不到了？别慌，这不是 bug，而是 PATH 的问题——还记得上一节讲的"黄页"吗？

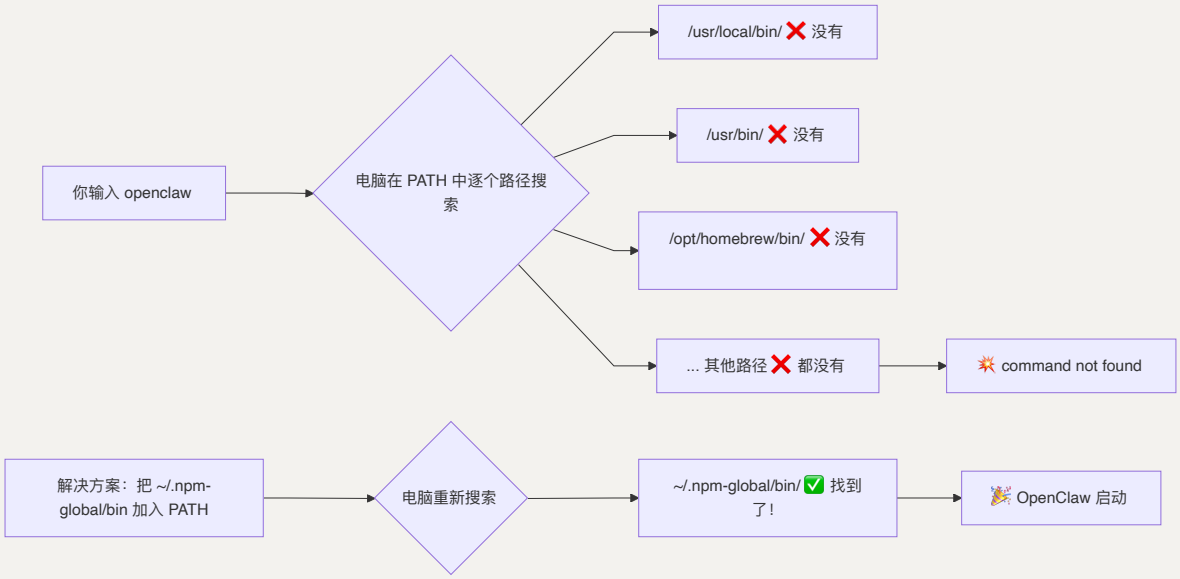
问题分析：为什么找不到？

npm 把 OpenClaw 安装到了哪里？我们来查一下：

```
$ npm prefix -g
/Users/hsail/.npm-global
```

所以 OpenClaw 的可执行文件在 `/Users/hsail/.npm-global/bin/openclaw`。但是你的 PATH 中有 `/Users/hsail/.npm-global/bin` 这个路径吗？回去看看 `echo $PATH` 的输出——**没有！这就是问题所在：程序装好了，但电脑不知道去哪里找它。**

用一张流程图来理解：



解决方案：配置 PATH

解决方案很简单——**把 npm 的全局安装路径添加到 PATH 中。**

执行以下命令：

```
# 把路径写入 ~/.zshrc（macOS 的 shell 配置文件）
$ echo 'export PATH="$HOME/.npm-global/bin:$PATH"' >> ~/.zshrc

# 让配置立即生效
$ source ~/.zshrc
```

这两条命令做了什么？

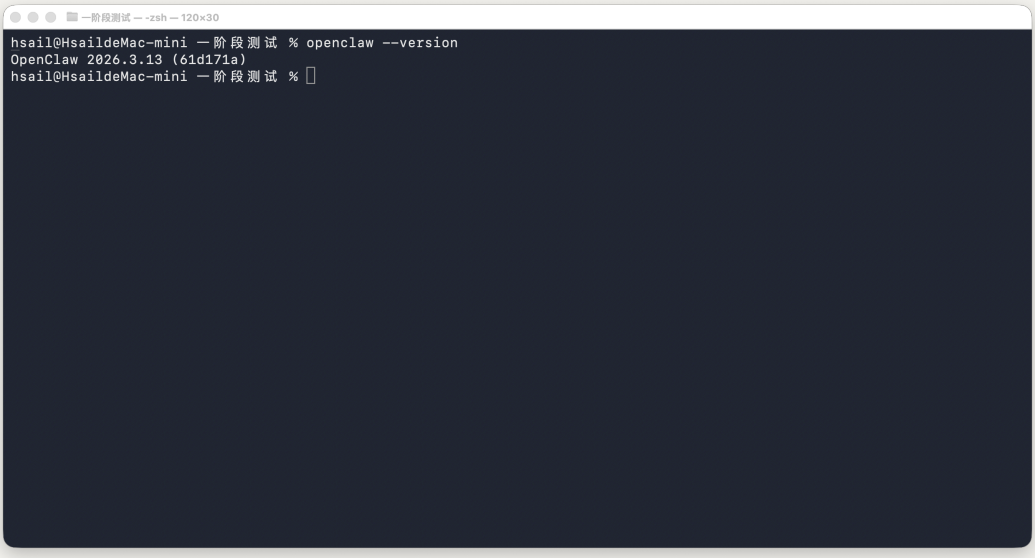
1. **`echo '...' >> ~/.zshrc`**：把一行配置追加到 `~/.zshrc` 文件末尾。`.zshrc` 是 macOS 终端的配置文件，每次打开终端都会自动读取它
2. **`source ~/.zshrc`**：让刚写入的配置在当前终端立即生效（否则需要关闭再重新打开终端）

> *小知识：`~` 是你的家目录的缩写，在 macOS 上通常是 `/Users/你的用户名`。`.zshrc` 文件名以 `.` 开头，表示它是一个隐藏文件，在 Finder 中默认看不到。*

验证安装成功

现在再试一次：

```
$ openclaw --version
OpenClaw 2026.3.13 (61d171a)
```

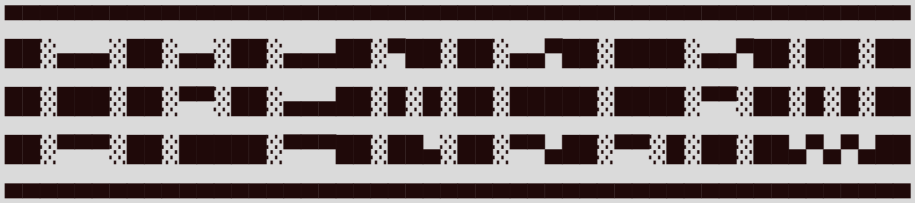
> 📸 *终端截图：成功显示 OpenClaw 版本号*
>
> 
>

成功了！ `OpenClaw 2026.3.13`——这就是目前的最新版本。括号里的 `61d171a` 是 Git commit 的短哈希值，可以忽略。

运行系统诊断

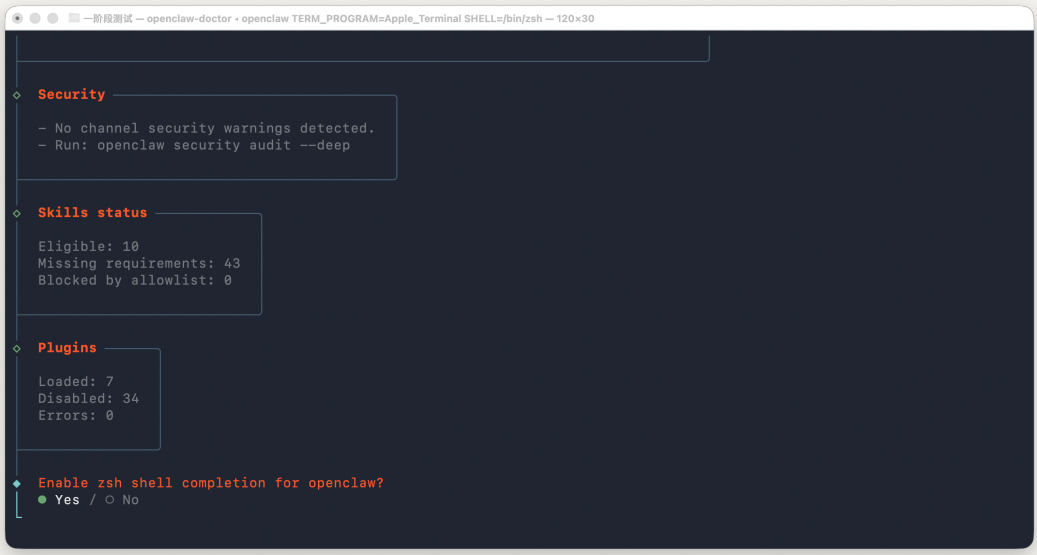
OpenClaw 自带一个"体检"命令 `openclaw doctor`，可以检查系统状态：

```
$ openclaw doctor
```



```
                    🦞 OPENCLAW 🦞

┌  OpenClaw doctor
│
◇  State integrity
│  - OAuth dir not present (~/.openclaw/credentials).
│    Skipping: no WhatsApp/pairing channel config active.
│
◇  Security
│  - No channel security warnings detected.
│  - Run: openclaw security audit --deep
│
◇  Skills status
│  Eligible: 8
│  Missing requirements: 45
│  Blocked by allowlist: 0
│
◇  Plugins
│  Loaded: 7
│  Disabled: 34
│  Errors: 0
│
◇  Gateway
│  Gateway not running.
│  Gateway target: ws://127.0.0.1:18789
│
└  Doctor complete.
```

> 📸 *终端截图：openclaw doctor 诊断结果*
>
> 
>

别被这些提示吓到——这些都是**正常的**。因为我们刚完成安装，还没有进行配置。来解读几个关键信息：

诊断项	状态	含义
State integrity	⚠️ OAuth dir not present	凭证目录还没创建，首次配置时自动生成
Security	✅ No warnings	安全检查通过，没有问题
Skills status	✅ 8 eligible	8 个内置技能已就绪
Plugins	✅ 7 loaded	7 个插件已加载
Gateway	⚠️ not running	Gateway 服务还没启动，下一章来做

核心信息就是：**OpenClaw 已经成功安装，等待首次配置**。配置的工作我们留到下一章。

补充：sharp 编译问题

如果你在安装过程中遇到了类似这样的报错：

```
npm ERR! sharp: Installation error: ...
npm ERR! sharp: Please try to install with:
SHARP_IGNORE_GLOBAL_LIBVIPS=1
```

这通常是因为你的 Mac 上通过 Homebrew 安装了 `libvips` 库，与 OpenClaw 的 `sharp` 模块（图像处理库）产生了冲突。

解决方案：

```
$ SHARP_IGNORE_GLOBAL_LIBVIPS=1 npm install -g openclaw@latest
```

在命令前面加上 `SHARP_IGNORE_GLOBAL_LIBVIPS=1` 环境变量，告诉 sharp 忽略系统级别的 libvips，使用自带的版本。